

SECURITY ASSESSMENT · BITCOIN CASH

FundTokens

Fixed-basket v1 contracts

Phase 1 findings, a second review of the remediated code, a formal verification finding, and verification of every fix through v0.1.0-rc18.

CLIENT	NETWORK	LANGUAGE
Fun(d)Tokens	Bitcoin Cash · CashTokens	CashScript 0.14
REVISIONS	**REPORT DATE**	**VERSION**
v0.1.0-rc2 → v0.1.0-rc18	5 November 2026	1.1

FINAL AUDIT REPORT

Contents

01 Executive summary 03
02 Scope and methodology 04
03 System overview 06
04 Findings 07
05 Conclusion 14

DOCUMENT

CLIENT	Fun(d)Tokens
PREPARED BY	Sighash Labs
VERSION	1.1
DATE	5 November 2026
CONTACT	info@sighash.xyz

VERSION HISTORY

VERSION	DATE	DESCRIPTION
Phase 1	20 Sep 2026	Findings, v0.1.0-rc2
1.0	26 Oct 2026	Final report, v0.1.0-rc17
1.1	05 Nov 2026	Adds FV-1, v0.1.0-rc18

01

Executive summary



Fun(d)Tokens engaged Sighash Labs to review the fixed-basket v1 smart contracts, which create, back, mint and redeem fund tokens on Bitcoin Cash.

Phase 1 assessed v0.1.0-rc2 and reported 25 findings (four Critical, four High, five Medium, twelve Low) and 15 informational notes; the most serious allowed unbacked issuance and outright reserve theft. The Fun(d)Tokens team reworked authorization and accounting in v0.1.0-rc16. We verified every Phase 1 entry against it and made a second pass, which found five Low issues, resolved in v0.1.0-rc17. A formal verification of v0.1.0-rc17 then found one more Critical issue, FV-1, which let anyone drain a single-reserve fund; the team fixed it in v0.1.0-rc18. We verified every fix.

0	9	31	15
Open findings at v0.1.0-rc18	Critical and High findings, all closed	Findings in total, all fixed, mitigated or acknowledged	Informational notes

Findings at v0.1.0-rc18



Assessment

At v0.1.0-rc18 the custody and issuance model holds. Authorization is bound to authenticated locking bytecode, every manager input sits inside a run the TransactionManager counts, and every recreated NFT pins its commitment. We re-derived the supply and reserve identities on every path we examined and found no way to take a public fund's backing or to obtain its tokens without depositing that backing.

02

Scope and methodology



2.1 Revisions

TAG	DATE	COMMIT	REVIEW
v0.1.0-rc2	04 Sep 2026	`be089b658edf753cd41f337b201cba2d50a702c5`	Phase 1
v0.1.0-rc16	07 Oct 2026	`d410b6c83705bb5f83971aa7519d58953cbe68e7`	Second pass
v0.1.0-rc17	18 Oct 2026	`10ffebb49437850e6f1b15d1bd838a0c755755fc`	Fix verification
v0.1.0-rc18	01 Nov 2026	`5ba3dc575c085425c7820daa0afde136dfd331b1`	Final

Repository github.com/FundTokens/FundTokens.Contracts. Toolchain at rc18: cashc 0.14.0-next.7, libauth 3.1.0-next.8, BCH-2026 rules.

2.2 Contracts in scope at v0.1.0-rc18

`fund-tokens-contracts/src/fund-types/fixed-basket/v1/contracts/`, directory tree `944244e6b6426b1eab326b1996742103da33b438`.

CONTRACT	FILE	ROLE	LINES
PER FUND			
TransactionManager	`manager.cash`	Fund threads; all mint and redeem accounting	285
FundManager	`fund.cash`	Unminted fund token supply	82
AssetManager	`asset.cash`	One reserve: the fund's BCH or one asset	74
SHARED BY EVERY FUND OF AN INSTANCE			
FundStartup	`startup.cash`	Validates a fund definition, creates its threads	132
FundInflowMint	`mint_inflow.cash`	Mints inflow threads	69
FundOutflowMint	`mint_outflow.cash`	Mints outflow threads	69
PublicFund	`public.cash`	Mints a public fund's supply, publishes its definition	141
FeeManager	`fee.cash`	Checks create and execute fee payments	128
PublicFundVault	`public_vault.cash`	Published definitions; proofs and delisting	135
AuthHeadVault	`authhead_vault.cash`	Public fund identities (BCMR authheads)	66
SimpleVault	`simple_vault.cash`	Fee vault	37
INSTANCE MAINTENANCE			
InstanceVault	`instance_vault.cash`	Instance parameters and lifecycle state	188
SimpleMinter	`simple_minter.cash`	Seeds the system minting NFTs	76
FeeMinter	`fee_minter.cash`	Issues fee tokens	77
authority	`lib/authority.cash`	Shared authorization check	32

15 files	1,591

ALSO REVIEWED

The fixed-basket v1 TypeScript builders and helpers, the v1 tests and the v1 documentation, where they bear on contract behaviour or on a finding.

OUT OF SCOPE

- The **bch-usd-target-blend** fund type, still in design
- The registry service, wallets and front ends
- Deployed genesis and instance transactions

2.3 Methodology

Line-by-line review of every contract function, with transaction topology re-derived from the code; a UTXO covenant hazard checklist covering output constraints, token identity and capability, encoding, time, value flow, liveness, deployment and accounting; and an invariant pass that restates every arithmetic line and tests the supply and reserve identities on every path. Every exploitable claim was backed by a proof of concept executed with libauth's BCH-2026 virtual machine and CashTokens validation of every input. Each fix was verified by re-running the original attack against the fixed revision. Our four second-pass proofs of concept behave as expected at v0.1.0-rc17 (the three attacks are rejected by the new checks; the AUD-002 regression still passes). For FV-1 we ran the team's regression test against both v0.1.0-rc17 and v0.1.0-rc18: the attack transactions are accepted at rc17 and rejected at rc18. At v0.1.0-rc18 the repository's full suite passes: 657 tests in 59 files, including the long-running stress suites.

CRITICAL	Direct total-loss or unbacked-issuance paths.
HIGH	Major authority escape, or permanent loss or lock of backing.
MEDIUM	Substantial integrity or availability failures, or privilege escalation with a restricted prerequisite.
LOW	Limited value loss, operational disruption, builder failures or configuration hazards.
INFORMATIONAL	Trust assumptions, hardening and documentation notes; not counted as findings.

Statuses: **Fixed**, the original attack is rejected; **Mitigated**, reduced to a bounded, documented residual; **Acknowledged**, kept by design and documented.

03

System overview

FundTokens lets anyone create a fund token: a CashToken on Bitcoin Cash backed by a fixed basket of BCH and other CashTokens per unit, held by contracts. Each fund gets a TransactionManager, a FundManager and one AssetManager per reserve, derived from its definition; shared contracts serve every fund of an instance.

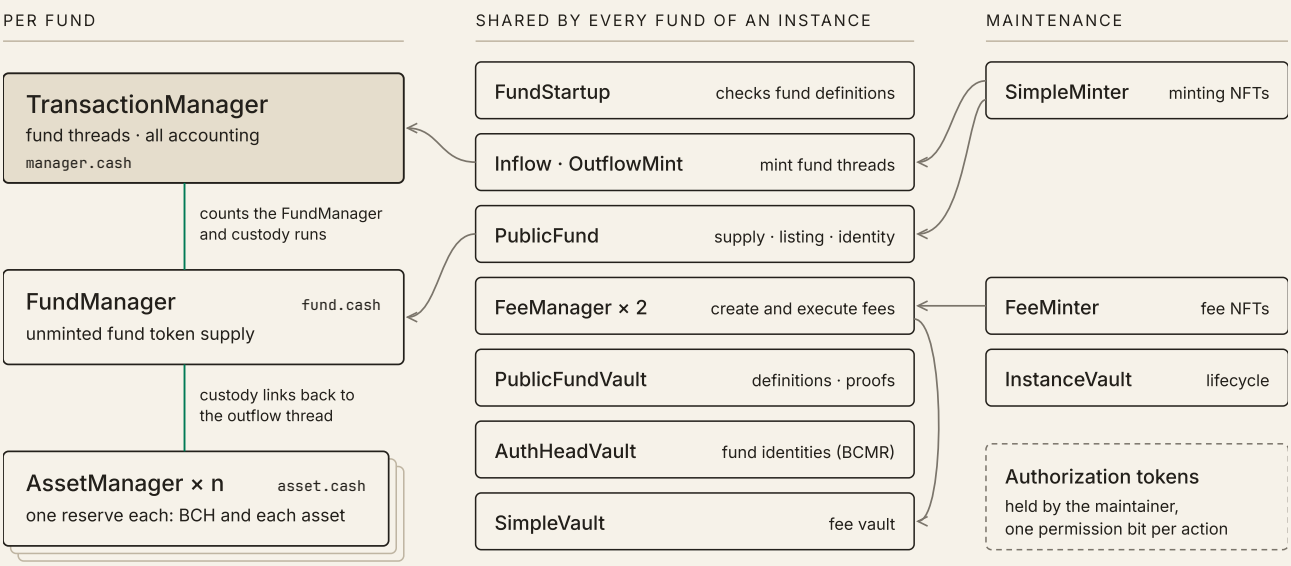


FIGURE 1 Fixed-basket v1 contract map. Arrows show what each contract creates or receives; every mint and redeem runs through the fund's TransactionManager.

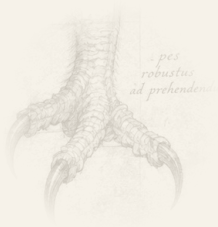
Create. One transaction spends a genesis output, whose transaction id becomes the fund's category. FundStartup checks the definition, the mint contracts create the fund's threads, and PublicFund mints the whole supply into the FundManager and publishes the definition to the PublicFundVault.

Mint and redeem. An operation spends a thread and the execute fee. The TransactionManager checks that exactly `units × fund.amount` fund tokens leave or return to the FundManager, and that exactly `units ×` the backing enters or leaves every reserve. Maintainers hold authorization tokens for fees, listings and instance state; none of them reaches a fund's backing.

04

Findings

Every finding, ordered by severity, with its resolution.



AUD-001 ■ CRITICAL ● FIXED RESOLVED IN RC16

Foreign-token substitution released unbacked fund tokens

ISSUE

Inflow summed token amounts without checking category, so foreign tokens returned to the FundManager shrank the measured release.

RESOLUTION

Every counted FundManager input and output must be tokenless or the fund category.

AUD-002 ■ CRITICAL ● FIXED RESOLVED IN RC16

Reserve inputs outside the accounted block enabled drains and fake deposits

ISSUE

A reserve input placed after a gap was released without the manager measuring it, draining reserves or faking deposits.

RESOLUTION

Each reserve input must follow the same AssetManager, its bound linked manager, or the fund's outflow thread, so every reserve sits in an accounted run. Our gap PoC is rejected in all three placements.

AUD-003 ■ CRITICAL ● FIXED RESOLVED IN RC16

Forged unlocking bytecode authorized reserve theft without a manager

ISSUE

The reserve chain compared attacker-supplied unlocking bytecode, so a crafted witness released reserves with no manager at all.

RESOLUTION

Only locking bytecode is compared; the forged-witness drains are rejected.

AUD-004 ■ CRITICAL ● FIXED RESOLVED IN RC16

A forged thread at an external address authorized fund-supply theft

ISSUE

Thread anchors checked category and commitment but not the TransactionManager script, so a forged thread released the whole supply.

RESOLUTION

The thread input must be locked by the fund's TransactionManager, which enforces full accounting.

FV-1 CRITICAL FIXED RESIDUAL OF AUD-002 · RESOLVED IN RC18

An empty-locking-bytecode input let anyone drain a single-reserve fund

ISSUE

An input with an empty locking bytecode left the AssetManager's "no lock recorded yet" marker looking unset, so a reserve outside the counted run could be released. In a single-reserve fund anyone could redeem two units and take the whole reserve.

RESOLUTION

The lock to match is now taken by position, from the input before the releasing one, so no value acts as a sentinel. Per-fund addresses change. The team's attack transactions are accepted at rc17 and rejected at rc18.

AUD-005 HIGH FIXED RESOLVED IN RC16

Startup return output permitted system-minter escape

ISSUE

The startup return was skipped by the anti-mint scans and could carry minting NFTs away.

RESOLUTION

The startup return must be tokenless.

AUD-006 HIGH FIXED RESOLVED IN RC17

Voluntary-fee output sent a rogue system minter to an attacker

ISSUE

The voluntary fee output was unconstrained and skipped by the creation scans.

RESOLUTION

rc16 pins the payment's destination and capability; rc17 closes the remaining immutable-NFT variant (R-2).

AUD-007 HIGH FIXED RESOLVED IN RC16

Token-bearing BCH deposits permanently locked backing

ISSUE

The BCH deposit could carry a token, making it unspendable by the BCH AssetManager.

RESOLUTION

The BCH custody deposit must be tokenless.

AUD-009 HIGH MITIGATED RESOLVED IN RC16

Large funds exceeded the redemption execution budget

ISSUE

Funds above about 39 assets accepted deposits they could not redeem.

RESOLUTION

Padding arguments buy operation budget and the library caps funds at 100 assets. The contracts still accept larger funds; this is documented.

AUD-010 MEDIUM FIXED RESOLVED IN RC16

Instance lifecycle state could be rewritten without authorization

ISSUE

`update()` accepted any state with no permission.

RESOLUTION

Permission `0x0040` is required, only defined states are accepted, and the vulnerable state is terminal.

AUD-011 MEDIUM MITIGATED RESOLVED IN RC16

Reserve fragmentation and backward scans capped redemption batches

ISSUE

Backward scans grew with input index, and zero-unit outflows could fragment reserves for free.

RESOLUTION

Only the head reserve scans; later reserves link in constant time and limits are measured. The one-unit residual is P2-3.

AUD-012 MEDIUM FIXED RESOLVED IN RC16

Unpinned thread commitments allowed permissionless thread destruction

ISSUE

Two funds' threads could swap commitments in one transaction and become unusable.

RESOLUTION

Thread and fee returns pin `nftCommitment`, and builders select threads by commitment.

AUD-013 MEDIUM FIXED RESOLVED IN RC16

Fee closure bypassed payment and exposed a minting slot

ISSUE

`close()` could recreate a FeeManager output that passed as proof of payment.

RESOLUTION

`close()` forbids any output at the FeeManager.

AUD-045 MEDIUM FIXED RESOLVED IN RC17

System token categories were never proven unique after genesis

ISSUE

Nothing let a client check that each system category was created as intended.

RESOLUTION

`verifyGenesis` checks every system category and the instance NFTs through the client's own node; the trust assumptions are documented.

AUD-008 LOW ACKNOWLEDGED RESOLVED IN RC16

Carrier BCH and attached NFTs on token reserves were unprotected

ISSUE

Token reserve accounting conserves fungible units only.

RESOLUTION

Accepted as policy: carrier BCH and attached NFTs go to the redeemer, and releasing custody requires redeeming at least one unit.

AUD-014 LOW ACKNOWLEDGED RESOLVED IN RC16

Unpinned recreation values permitted operational BCH depletion

ISSUE

Recreated outputs preserved script and token state but not satoshi value.

RESOLUTION

Documented as design.

AUD-015 LOW FIXED RESOLVED IN RC16

Outflow loops could read beyond transaction bounds

ISSUE

Runs reaching the last input or output aborted.

RESOLUTION

Loops are bounded; the remaining unbounded loop fails closed and is documented.

AUD-018 LOW FIXED RESOLVED IN RC16

Documented mutable authorization tokens did not match permission checks

ISSUE

Documentation described a mutable authorization form the contracts reject.

RESOLUTION

Documentation states that only immutable authorization NFTs count.

AUD-019 LOW MITIGATED RESOLVED IN RC16

Creation accepted conflicting or unusable fund categories

ISSUE

Zero, system or self-referencing categories were accepted.

RESOLUTION

The library rejects them; the contracts still accept them, which is documented.

AUD-020 LOW FIXED RESOLVED IN RC16

Minimum units and output policy were not validated consistently

ISSUE

Builders could produce sub-dust outputs.

RESOLUTION

Fixed in the library.

AUD-022 LOW FIXED RESOLVED IN RC16

Voluntary fee NFTs crashed reference fee selection

ISSUE

Decoding a voluntary fee threw.

RESOLUTION

The library skips voluntary and malformed fee NFTs.

AUD-023 LOW FIXED RESOLVED IN RC16

Public-fund chunk outputs admitted fungible tokens

ISSUE

Chunk outputs did not constrain token amount.

RESOLUTION

Every chunk must have amount 0, so every chunk is an NFT.

AUD-025 LOW FIXED RESOLVED IN RC16

Complete-burn scans missed capability-bearing variants

ISSUE

Burn scans compared the full category, missing mutable and minting variants.

RESOLUTION

Scans compare the first 32 bytes of each category.

AUD-026 LOW FIXED RESOLVED IN RC16

Insufficient reserves produced malformed builds

ISSUE

Builders misaligned change or indexed missing UTXOs.

RESOLUTION

The library raises clear errors before building.

AUD-027 LOW FIXED RESOLVED IN RC17

An authorization token could authorize its own theft

ISSUE

A misplaced authorization NFT satisfied the check for its own spend.

RESOLUTION

rc16 ignores authority at the caller's own address; rc17 closes the FeeManager variant (R-1).

AUD-046 LOW ACKNOWLEDGED RESOLVED IN RC16

Instance proof covers two inputs, leaving a surplus data token unconstrained

ISSUE

A third instance UTXO from the creating transaction would be unconstrained.

RESOLUTION

The two-UTXO layout is a documented deployment assumption, and `verifyGenesis` now checks it.

P2-1 LOW FIXED RESOLVED IN RC17

Listing proofs were not anchored to the first chunk of a broadcast

ISSUE

`PublicFundVault.proof()` accepted a run of chunks starting at any chunk. A fund creator could choose the chunk length and asset bytes so that a later chunk of a genuine listing began a second valid listing, for a fund whose genesis and supply were never checked. Rated Low because the registry reads each listing from its creation transaction, not from a vault proof.

RESOLUTION

Every chunk after the first now starts with a continuation byte `0x03`, enforced by `broadcast()` and by `data()`, and every chunk but the last is a full 128 bytes. A proof started from a later chunk reads `0x03` as its type and fails. `proof()` and `burn()` also take padding, so large listings can be proven and delisted.

R-1 LOW FIXED RESIDUAL OF AUD-027 · RESOLVED IN RC17

An authorization NFT held by a FeeManager could be borrowed by anyone

ISSUE

The default branch of `FeeManager.pay()` returned any token unchanged. An authorization NFT sent to a FeeManager by mistake could be spent there while another contract's gated function accepted it, giving anyone every permission it carried.

RESOLUTION

The default branch requires a tokenless input. A stray token stays put until the steward recovers it with `close()`, and the library only offers immutable fee NFTs.

R-2 LOW FIXED RESIDUAL OF AUD-006 · RESOLVED IN RC17

The voluntary fee slot accepted immutable system-category NFTs

ISSUE

Both creation scans skipped the create-fee payment output, and a voluntary fee let it carry any immutable NFT. Fund creation could mint a forged thread or publicFund chunk there; the chunk could then be proven at the PublicFundVault.

RESOLUTION

`start()` rejects inflow and outflow categories on the fee payment, and `broadcast()` rejects the fund category and the publicFund category.

P2-2 LOW FIXED RESOLVED IN RC17

Fund supply fragmentation priced the mint builder out

ISSUE

A cheap mint could split the remaining supply into about 1,000 one-token UTXOs. `addInflow()` picked supply at random and did not check size, so most later mints exceeded the standard transaction size.

RESOLUTION

The builder takes one supply UTXO that covers the mint, or the largest first, and refuses an oversized mint with the number of units that fit. The contracts are unchanged.

P2-3 LOW ACKNOWLEDGED RESIDUAL OF AUD-011 · RESOLVED IN RC17

Redemption change could split custody into many UTXOs

ISSUE

A one-unit redemption could return its change as any number of dust custody outputs, fragmenting reserves at little cost.

RESOLUTION

Accepted by design and documented (`CONTRACTS.md`, “Fragmentation is griefing, not loss”). Nothing is lost or locked, later operations consolidate, and the builders refuse transactions too large to relay. Limiting change outputs would add cost to every redemption.

4.1 Informational notes

ID	NOTE	STATUS	RESOLUTION
AUD-016	Fungible-only fee tokens are unsupported	ACKNOWLEDGED	Fails closed.
AUD-017	Protocol addresses have no general recovery path	ACKNOWLEDGED	Documented; the FeeManager exception was removed in rc17 (R-1).
AUD-021	Shared UTXOs need transaction coordination	ACKNOWLEDGED	Inherent to the design.
AUD-028	NFT recreation did not preserve fungible amounts	FIXED	Every return pins the amount.
AUD-029	Authorization parsing omits role and length checks	ACKNOWLEDGED	Documented; short commitments fail closed.
AUD-030	Thread derivation assumes fixed push sizes	ACKNOWLEDGED	Holds at rc17: templates of 120 and 152 bytes.

AUD-031	Administrative authority was understated	● FIXED	Every power is documented.
AUD-032	Private funds require issuer trust	● FIXED	Documented, with `verifyFundGenesis` (rc17).
AUD-033	Best-fee sorting used a boolean comparator	● FIXED	Numeric minimum.
AUD-034	Documentation diverged from the code	● FIXED	Rewritten; `burn()` enforces zero value (rc17).
AUD-035	Registry parameters rely on transport	⊖ ACKNOWLEDGED	Documented; `verifyGenesis` checks them (rc17).
AUD-037	Redemption conditionally skips a category check	⊖ ACKNOWLEDGED	Contained by the manager.
AUD-038	Serialization truncated oversized satoshi values	● FIXED	Validated in the library.
AUD-041	Asset ordering was locale-sensitive	● FIXED	Compared as 256-bit integers.
AUD-044	Fee category used an ambiguous numeric zero	● FIXED	Compared as 32 zero bytes; dead check removed (rc17).

05

Conclusion

The remediation across v0.1.0-rc16, v0.1.0-rc17 and v0.1.0-rc18 is thorough. Every Critical and High finding from Phase 1 is fixed or, for AUD-009, mitigated and documented, every second-pass finding is fixed or accepted with a documented rationale, and the formal verification finding FV-1 is fixed. No finding remains open at v0.1.0-rc18, and we found no way to take a public fund's backing or to obtain its tokens without depositing that backing.

Five findings remain by design and are documented: carrier value on token reserves (AUD-008), unpinned recreation values (AUD-014), the instance NFT layout (AUD-046), and two forms of UTXO fragmentation that cost the attacker more than the defender (AUD-011, P2-3). None of them puts backing at risk.

The FV-1 fix changes the AssetManager bytecode and with it every per-fund address, so funds must be deployed from the v0.1.0-rc18 artifacts.

5.1 Disclaimer

This report reflects a best-effort, point-in-time review of the FundTokens fixed-basket v1 contracts at the revisions listed in Section 2. It does not constitute a warranty of fitness, correctness or absence of vulnerabilities, and cannot certify the complete security of the system. Independent review, ongoing testing and a public bug bounty are recommended.

Smart contracts depend on their blockchain, virtual machine, compilers, libraries and tooling, and on external services such as the registry and wallets; these are outside the scope of this audit. Code outside the scope in Section 2, including the bch-usd-target-blend fund type, was not reviewed, and changes after v0.1.0-rc18 are not covered.

No responsibility is accepted for any loss of funds or damages arising from the use, deployment or operation of the audited contracts.

SIGHASH LABS

We design, build and audit smart contracts and transaction protocols for UTXO systems.

sighash.xyz info@sighash.xyz x.com/SighashLabs